



Database Systems and Web (15B11CI312)

Database Systems and Web

Lecture 10: EER

Contents to be covered

- Enhanced ER Modelling

- Subclass

- Superclass

- Generalization

- Specialization

The Enhanced Entity-Relationship (EER) Model

Enhanced ER (EER) model

- Created to design more accurate database schemas
 - Reflect the data properties and constraints more precisely
- More complex requirements than traditional applications

Subclasses, Superclasses, and Inheritance

EER model includes all modeling concepts of the ER model

In addition, EER includes:

- **Subclasses** and **superclasses**
- **Specialization** and **generalization**
- **Category** or **union type**
- **Attribute** and **relationship inheritance**

Subclasses and Superclasses

An entity type may have additional meaningful subgroupings of its entities

- Example: EMPLOYEE may be further grouped into:
 - SECRETARY, ENGINEER, TECHNICIAN, ...
 - Based on the EMPLOYEE's Job
 - MANAGER
 - EMPLOYEEs who are managers
 - SALARIED_EMPLOYEE, HOURLY_EMPLOYEE
 - Based on the EMPLOYEE's method of pay

EER diagrams extend ER diagrams to represent these additional subgroupings, called *subclasses* or *subtypes*

Subclasses and Superclasses

Each of these subgroupings is a subset of EMPLOYEE entities

Each is called a subclass of EMPLOYEE

EMPLOYEE is the superclass for each of these subclasses

These are called superclass/subclass relationships:

- EMPLOYEE/SECRETARY
- EMPLOYEE/TECHNICIAN
- EMPLOYEE/MANAGER
- ...

Subclasses and Superclasses

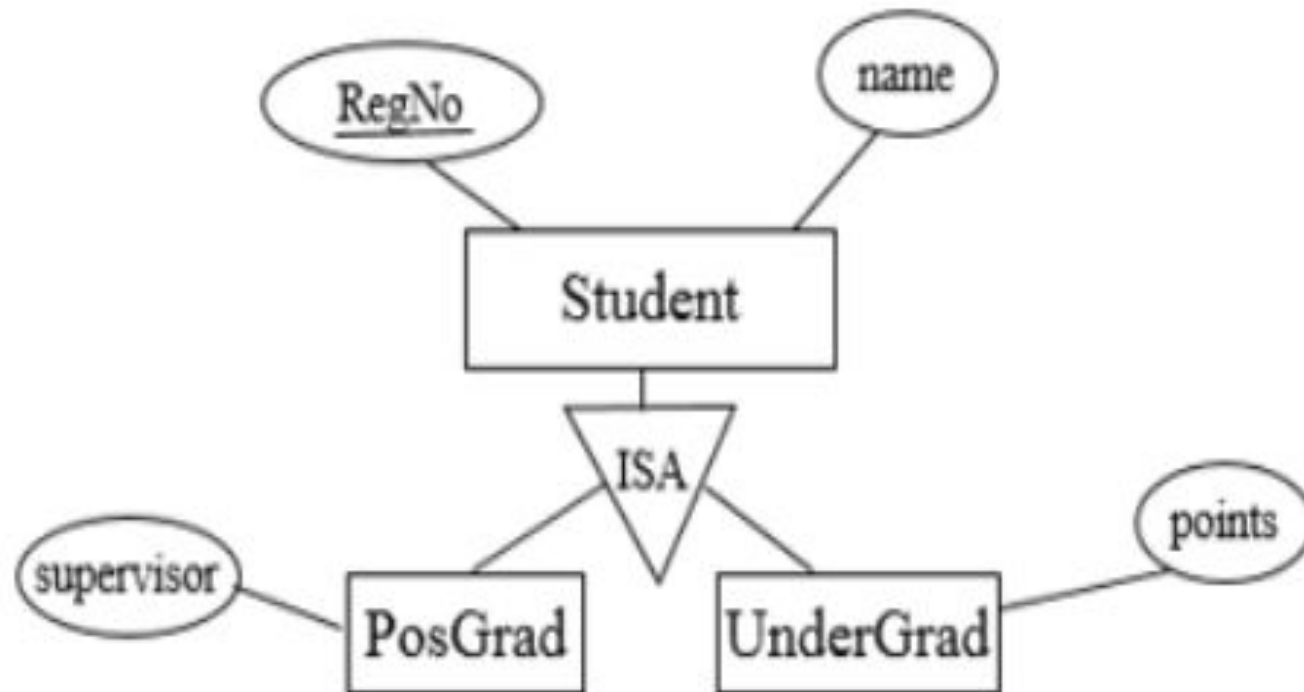
These are also called **IS-A relationships**

- SECRETARY IS-A EMPLOYEE, TECHNICIAN IS-A EMPLOYEE,

Note: An entity that is member of a subclass represents the same real-world entity as some member of the superclass:

- **The subclass member is the same entity in a *distinct specific role***
- An entity cannot exist in the database merely by being a member of a subclass; it must also be a member of the superclass
- A member of the superclass can be optionally included as a member of any number of its subclasses

IS-A Relationship



Attribute Inheritance in Superclass / Subclass Relationships

An entity that is member of a subclass *inherits*

- All attributes of the entity as a member of the superclass
- All relationships of the entity as a member of the superclass

Example:

- In the previous slide, SECRETARY (as well as TECHNICIAN and ENGINEER) inherit the attributes Name, SSN, ..., from EMPLOYEE
- Every SECRETARY entity will have values for the inherited attributes

Membership Constraints

Predicate defined subclasses

- ~~The subclass is defined through a predicate on the attributes of the superclass~~
- Display a predicate-defined subclass by writing the predicate condition next to the line attaching the subclass to its superclass

Attribute defined subclasses

- The subclasses in the specialization are all defined by the same attribute of the superclass
 - Attribute is called the defining attribute of the specialization
 - Example: JobType is the defining attribute of the specialization {SECRETARY, TECHNICIAN, ENGINEER} of EMPLOYEE

User defined subclasses

- Membership in the subclasses is determined at the insertion operation level

Specialization and Generalization

Specialization

Specialization is the process of defining a set of subclasses of a superclass

The set of subclasses is based upon some distinguishing characteristics of the entities in the superclass

- Example: {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of EMPLOYEE based upon *job type*.
 - May have several specializations of the same superclass

Subclass can define:

- **Specific attributes**
- **Specific relationship types**

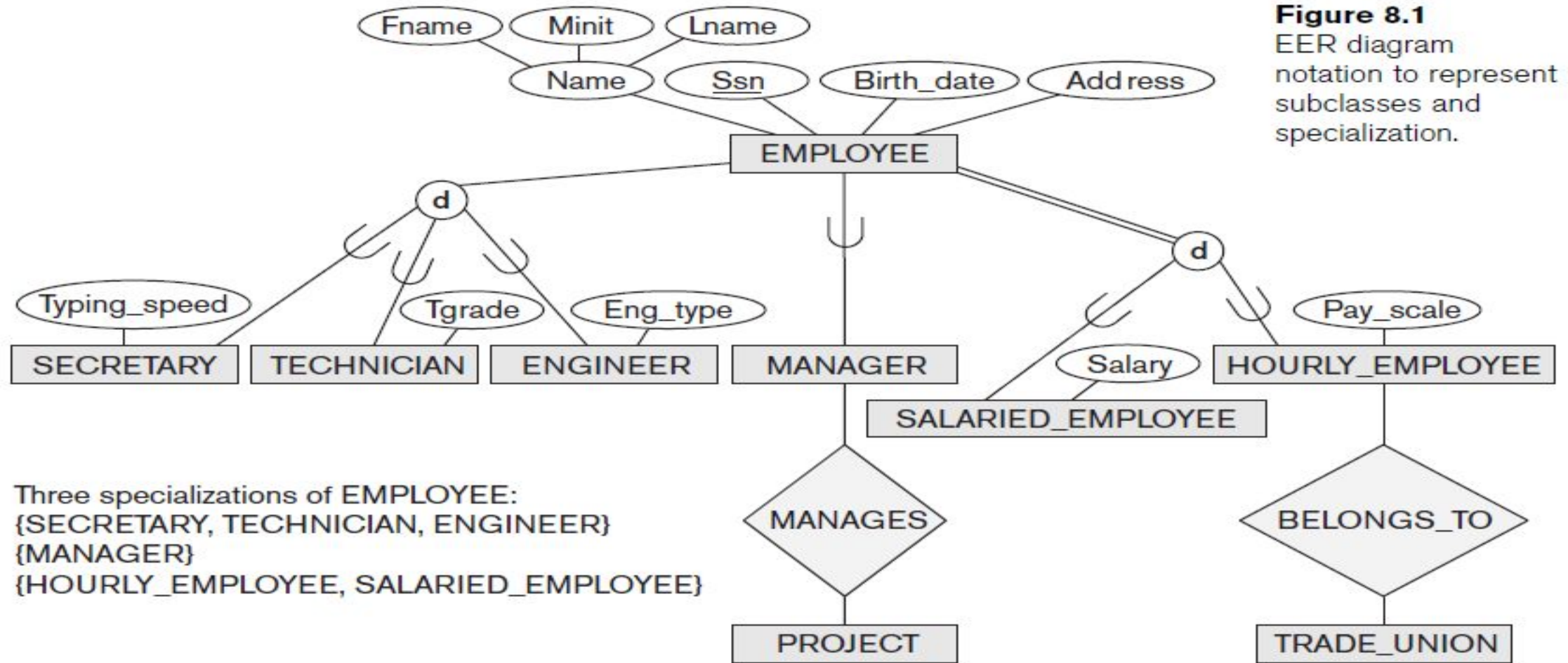


Figure 8.1
EER diagram
notation to represent
subclasses and
specialization.

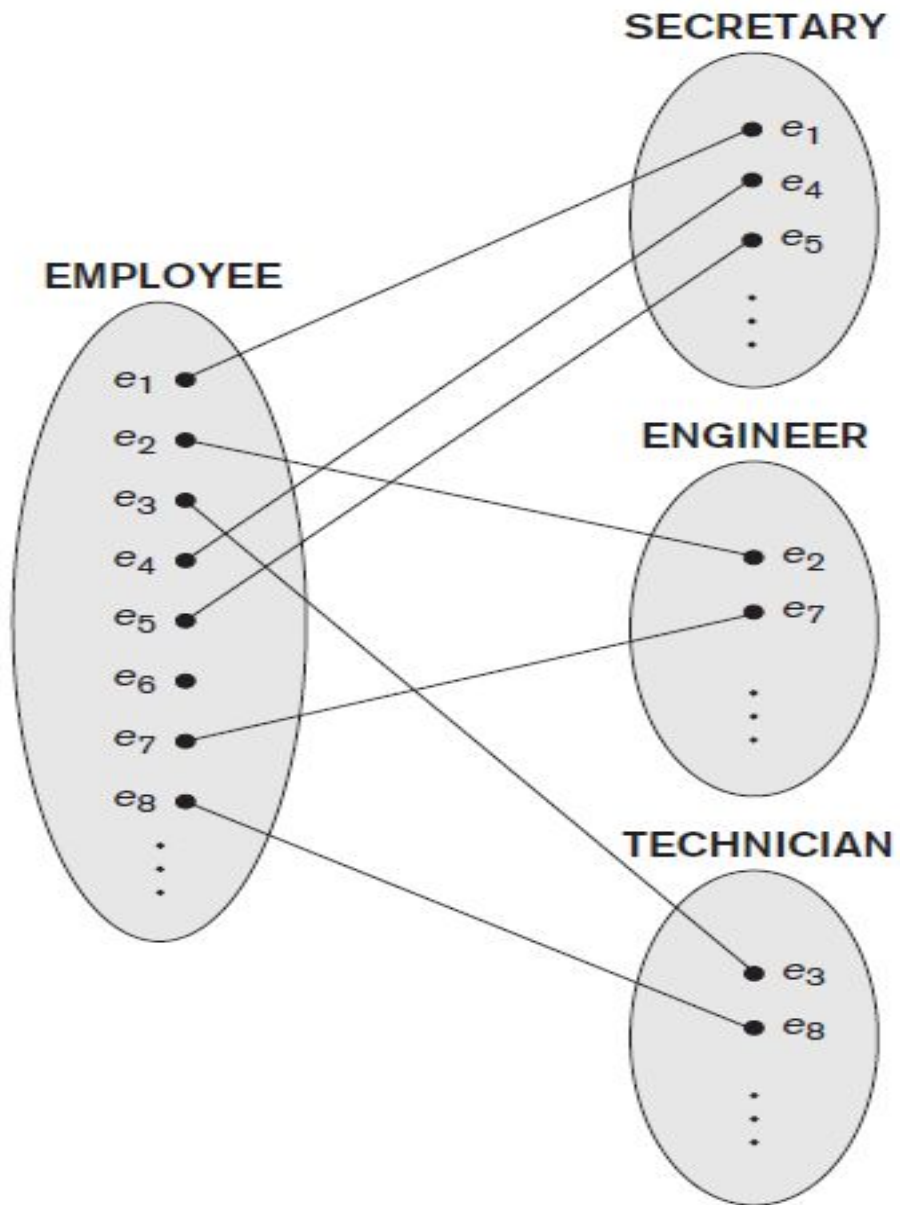


Figure 8.2
Instances of a specialization.

Generalization

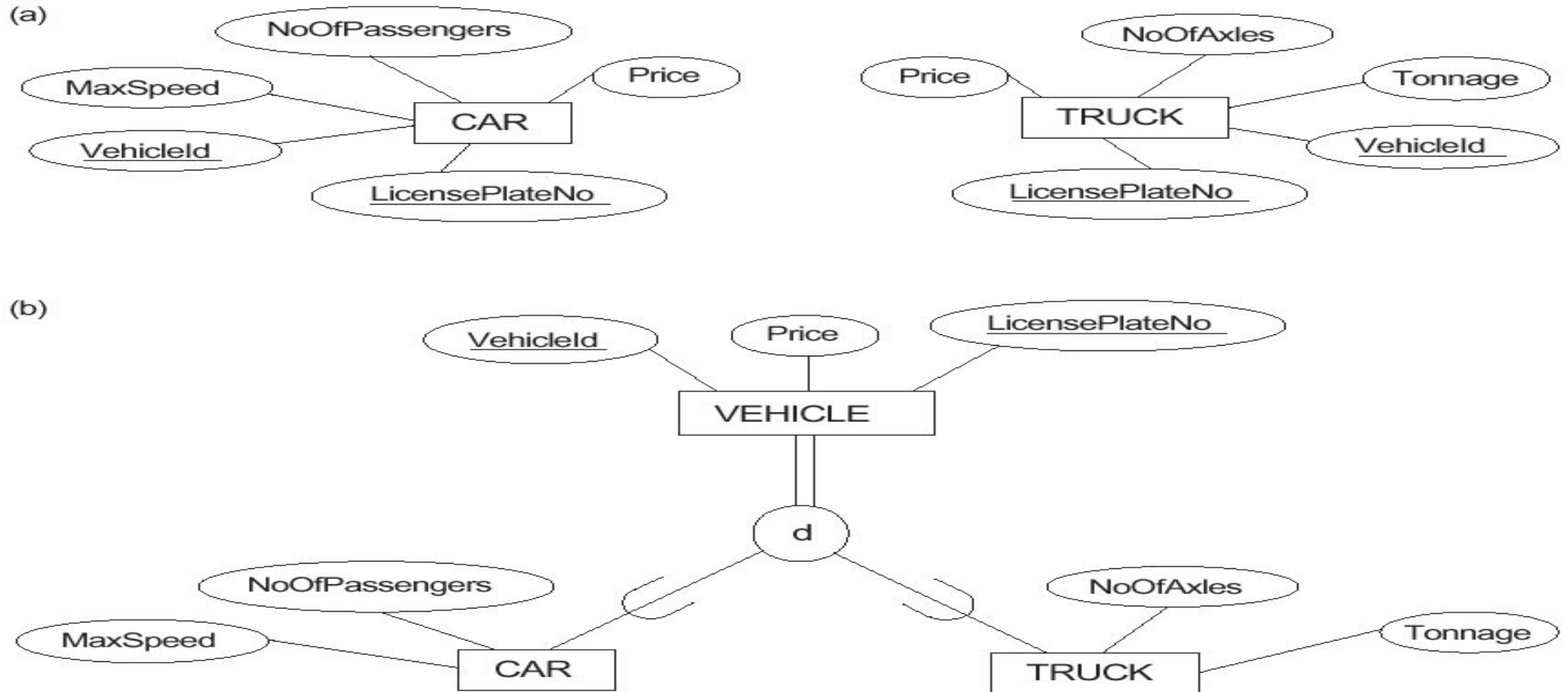
Generalize into a single **superclass**

- Original entity types are special subclasses

Generalization

- Process of defining a generalized entity type from the given entity types

Figure 4.3 Examples of generalization. (a) Two entity types CAR and TRUCK. (b) Generalizing CAR and TRUCK into VEHICLE.



Constraints on Specialization and Generalization

Two basic constraints can apply to a specialization/generalization:

- Disjointness Constraint:
- Completeness Constraint:

Disjointness Constraints

Specifies that the subclasses of the specialization must be *disjoint*:

- an entity can be a member of at most one of the subclasses of the specialization
- Specified by **d** in EER diagram
- If not disjoint, specialization is *overlapping*:
 - that is the same entity may be a member of more than one subclass of the specialization
- Specified by **o** in EER diagram

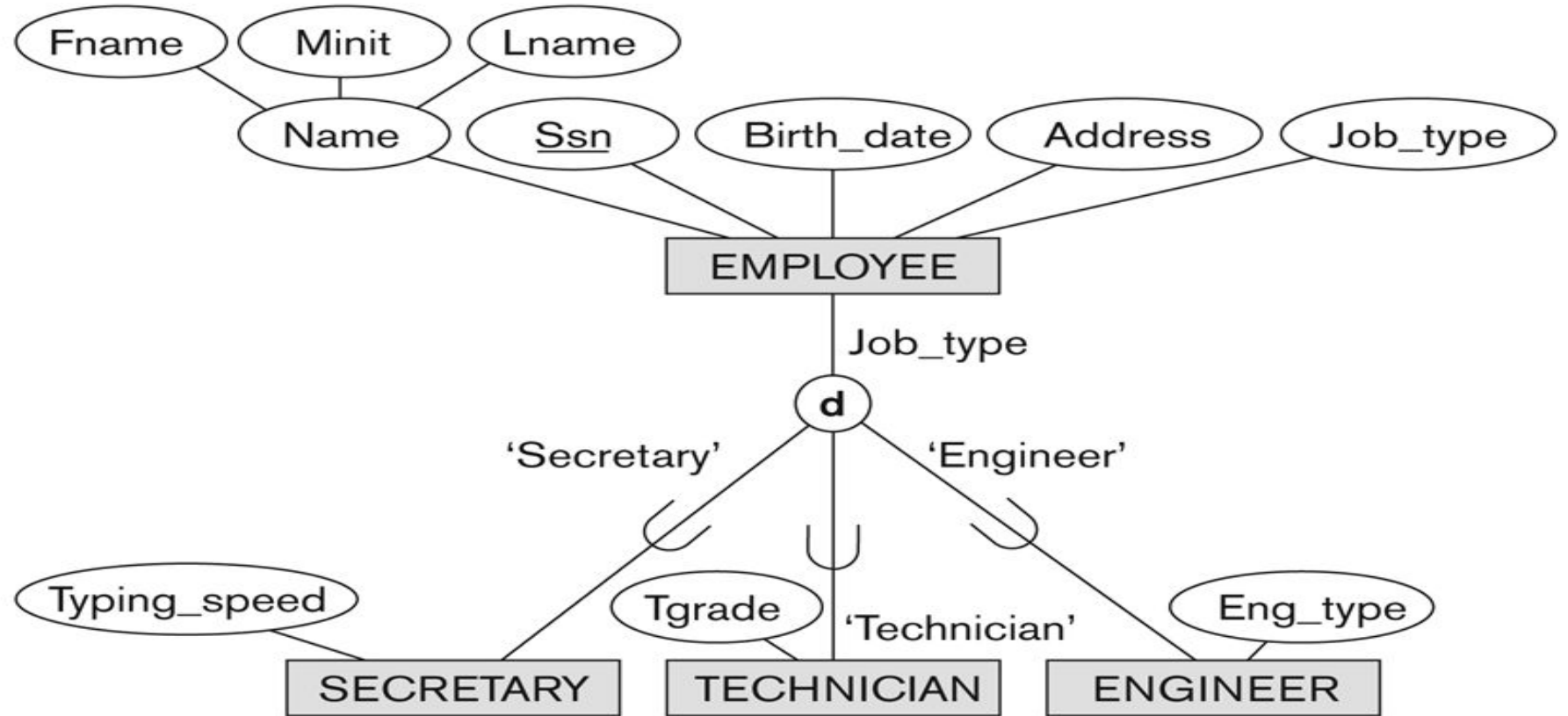
Completeness Constraint:

- *Total* specifies that every entity in the superclass must be a member of some subclass in the specialization/generalization
- Shown in EER diagrams by a **double line**
- *Partial* allows an entity not to belong to any of the subclasses
- Shown in EER diagrams by a single line

Example of disjoint partial Specialization

Figure 4.4

EER diagram notation for an attribute-defined specialization on Job_type.



Example of overlapping total Specialization

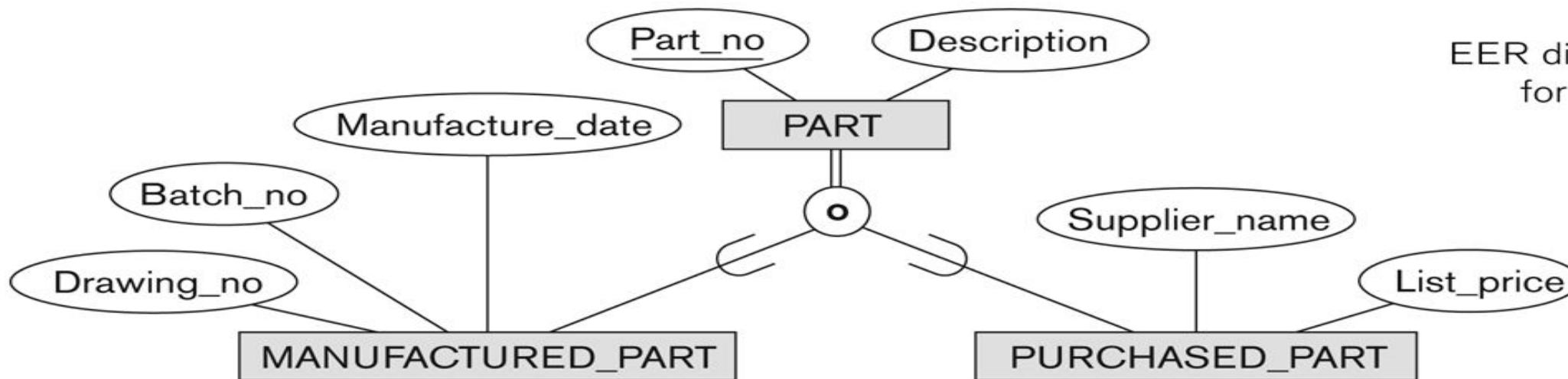


Figure 4.5
EER diagram notation
for an overlapping
(nondisjoint)
specialization.

Structures in Specialization

Multiple Specializations

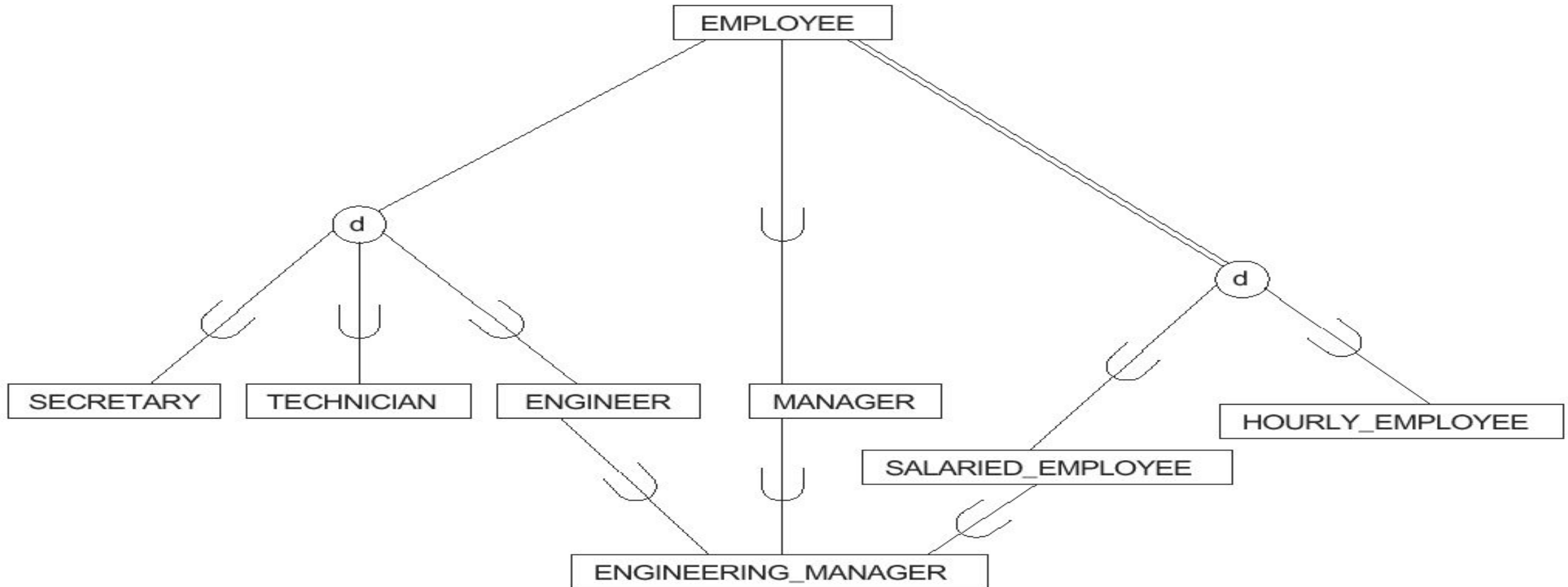
Specialization Hierarchy

- *Hierarchy* has a constraint that every subclass has only one superclass (called *single inheritance*); this is basically a *tree structure*

Lattice Specializations

- A subclass may belong to more than one class (called *multiple inheritance*)

Figure 4.6 A specialization lattice with the shared subclass ENGINEERING_MANAGER.



Source: *Fundamentals of database systems* / Ramez Elmasri, Shamkant B. Navathe.—6th ed, Pearson Publications

Specialization / Generalization Lattice Example (UNIVERSITY)

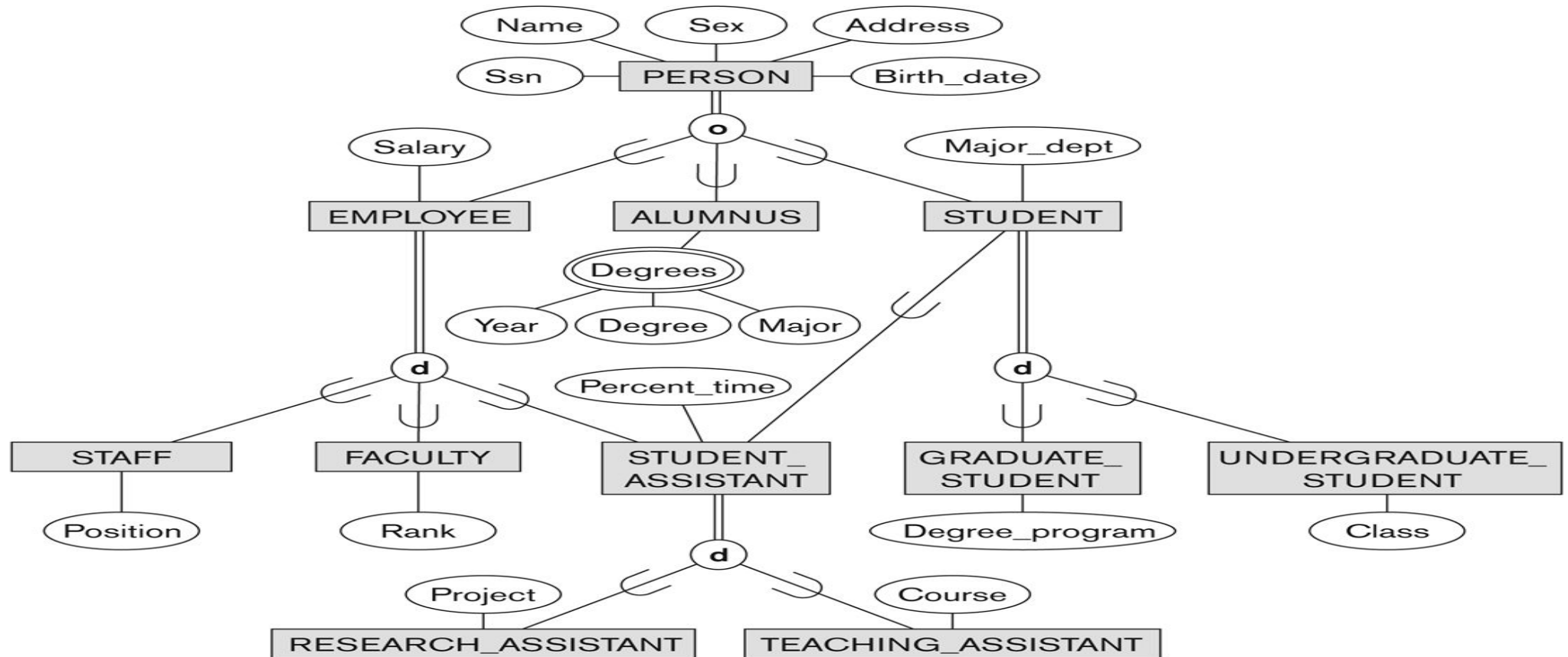


Figure 4.7

A specialization lattice with multiple inheritance for a UNIVERSITY database.

UNION

Figure 4.8 An illustration of how to represent the UNION of two or more entity types/classes using the category notation. Two categories are shown: OWNER and REGISTERED_VEHICLE.

